

Another method to define default values for model fields is using database-computed default values. Introduced in Django 5, this feature allows you to use underlying database functions to generate default values. For instance, the following code uses the database server's current date and time as the default for the publish field:

```
from django.db import models
from django.db.models.functions import Now

class Post(models.Model):
    # ...
    publish = models.DateTimeField(db_default=Now())
```

```
-----

from django.db import models
from django.utils import timezone

# Create your models here.
class Post(models.Model):
    class Status(models.TextChoices):
        DRAFT = 'DF', 'Draft'
        PUBLISHED = 'PB', 'Published'

    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    body = models.TextField()
    publish = models.DateTimeField(default=timezone.now)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)
    status = models.CharField(max_length=2, choices=Status, default=Status.DRAFT)
    class Meta:
        ordering = ['-publish']
        indexes = [
            models.Index(fields=['-publish'])
        ]
```

```
def __str__(self):
    return self.title
```

```
(.venv) C:\Tests\j-l-django\Test-02\jlokastite>python manage.py shell
13 objects imported automatically (use -v 2 for details).
```

```
Ctrl click to launch VS Code Native REPL
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr  8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from blog.models import Post
>>> Post.Status.choices
[('DF', 'Draft'), ('PB', 'Published')]
>>> Post.Status.values
['DF', 'PB']
>>> Post.Status.labels
['Draft', 'Published']
>>> Post.Status.names
['DRAFT', 'PUBLISHED']
```

```
(.venv) C:\Tests\j-l-django\Test-02\jlokastite>python manage.py sqlmigrate blog 0001
--
-- Create model Post
--
CREATE TABLE `blog_post` (`id` bigint AUTO_INCREMENT NOT NULL PRIMARY KEY, `title` varchar(250) NOT NULL, `slug` varchar(250) NOT NULL, `body` longtext NOT NULL, `publish` datetime(6) NOT NULL, `created` datetime(6) NOT NULL, `updated` datetime(6) NOT NULL, `status` varchar(2) NOT NULL, `author_id` integer NOT NULL);
ALTER TABLE `blog_post` ADD CONSTRAINT `blog_post_author_id_dd7a8485_fk_auth_user_id` FOREIGN KEY (`author_id`) REFERENCES `auth_user` (`id`);
CREATE INDEX `blog_post_slug_b95473f2` ON `blog_post` (`slug`);
CREATE INDEX `blog_post_publish_bb7600_idx` ON `blog_post` (`publish` DESC);
```

To register in admin site:

```
from django.contrib import admin
from .models import Post
# Register your models here.
# admin.site.register(Post)
@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ['title', 'slug', 'author', 'publish', 'status']
    list_filter = ['status', 'created', 'publish', 'author']
    search_fields = ['title', 'body']
    prepopulated_fields = {'slug': ('title',)}
    raw_id_fields = ['author']
    date_hierarchy = 'publish'
    ordering = ['status', 'publish']
    show_facets = admin.ShowFacets.ALWAYS
    *****
```

This QuerySet will return all posts with the exact title *Who was Django Reinhardt?*. Let's review the SQL statement generated with this QuerySet. Run the following code in the shell:

```
>>> posts = Post.objects.filter(title='Who was Django Reinhardt?')
>>> print(posts.query)
```

By printing the query attribute of the QuerySet, we can get the SQL produced by it:

```
SELECT "blog_post"."id", "blog_post"."title", "blog_post"."slug", "blog_
post"."author_id", "blog_post"."body", "blog_post"."publish", "blog_
post"."created", "blog_post"."updated", "blog_post"."status" FROM "blog_post"
WHERE "blog_post"."title" = Who was Django Reinhardt? ORDER BY "blog_
post"."publish" DESC
```

The following example shows the greater than (gt) lookup:

```
>>> Post.objects.filter(id__gt=3)
```

The equivalent SQL clause is `WHERE ID > 3`.

This example shows the greater than or equal to lookup:

```
>>> Post.objects.filter(id__gte=3)
```

This one shows the less than lookup:

```
>>> Post.objects.filter(id__lt=3)
```

This shows the less than or equal to lookup:

```
>>> Post.objects.filter(id__lte=3)
```

A case-sensitive/insensitive starts-with lookup can be performed with the `startswith` and `istartswith` lookup types, respectively:

```
>>> Post.objects.filter(title__startswith='who')
```

A case-sensitive/insensitive ends-with lookup can be performed with the `endswith` and `iendswith` lookup types, respectively:

```
>>> Post.objects.filter(title__iendswith='reinhardt?')
```

Activate
Go to Settings

There are also different lookup types for date lookups. An exact date lookup can be performed as follows:

```
>>> from datetime import date
>>> Post.objects.filter(publish__date=date(2024, 1, 31))
```

This shows how to filter a `DateField` or `DateTimeField` field by year:

```
>>> Post.objects.filter(publish__year=2024)
```

You can also filter by month:

```
>>> Post.objects.filter(publish__month=1)
```

And you can filter by day:

```
>>> Post.objects.filter(publish__day=1)
```

You can chain additional lookups to date, year, month, and day. For example, here is a lookup for a value greater than a given date:

```
>>> Post.objects.filter(publish__date__gt=date(2024, 1, 1))
```

To lookup related object fields, you also use the two-underscores notation. For example, to retrieve the posts written by the user with the admin username, use the following:

```
>>> Post.objects.filter(author__username='admin')
```

You can also chain additional lookups for the related fields. For example, to retrieve posts written by any user with a username that starts with ad, use the following:

```
>>> Post.objects.filter(author__username__startswith='ad')
```

You can also filter by multiple fields. For example, the following QuerySet retrieves all posts published in 2024 by the author with the username admin:

```
>>> Post.objects.filter(publish__year=2024, author__username='admin')
```

Chaining filters

The result of a filtered QuerySet is another QuerySet object. This allows you to chain QuerySets together. You can build an equivalent QuerySet to the previous one by chaining multiple filters:

```
>>> Post.objects.filter(publish__year=2024) \
>>>     .filter(author__username='admin')
```

Excluding objects

You can exclude certain results from your QuerySet by using the `exclude()` method of the manager. For example, you can retrieve all posts published in 2024 whose titles don't start with `Why`:

```
>>> Post.objects.filter(publish__year=2024) \
>>>     .exclude(title__startswith='Why')
```

Ordering objects

The default order is defined in the `ordering` option of the model's `Meta`. You can override the default ordering using the `order_by()` method of the manager. For example, you can retrieve all objects ordered by their title, as follows:

```
>>> Post.objects.order_by('title')
```

Ascending order is implied. You can indicate descending order with a negative sign prefix, like this:

```
>>> Post.objects.order_by('-title')
```

You can order by multiple fields. The following example orders objects by author first and then title:

```
>>> Post.objects.order_by('author', 'title')
```

To order randomly, use the string `'?'`, as follows:

```
>>> Post.objects.order_by('?')
```

Limiting QuerySets

You can limit a `QuerySet` to a certain number of results by using a subset of Python's array-slicing syntax. For example, the following `QuerySet` limits the results to 5 objects:

```
>>> Post.objects.all()[:5]
```

This translates to a SQL `LIMIT 5` clause. Note that negative indexing is not supported.

```
>>> Post.objects.all()[3:6]
```

The preceding translates to a SQL `OFFSET 3 LIMIT 6` clause, to return the fourth through sixth objects.

To retrieve a single object, you can use an index instead of a slice. For example, use the following to retrieve the first object of posts in random order:

```
>>> Post.objects.order_by('?')[0]
```

Counting objects

The `count()` method counts the total number of objects matching the `QuerySet` and returns an integer. This method translates to a `SELECT COUNT(*)` SQL statement. The following example returns the total number of posts with an `id` lower than 3:

```
>>> Post.objects.filter(id_lt=3).count()
2
```

Checking if an object exists

The `exists()` method allows you to check if a `QuerySet` contains any results. This method returns `True` if the `QuerySet` contains any items and `False` otherwise. For example, you can check if there are any posts with a title that starts with *Why* using the following `QuerySet`:

```
>>> Post.objects.filter(title__startswith='Why').exists()
False
```

Activate Windows

Deleting objects

If you want to delete an object, you can do it from an object instance using the `delete()` method, as follows:

```
>>> post = Post.objects.get(id=1)
>>> post.delete()
```

Note that deleting objects will also delete any dependent relationships for `ForeignKey` objects defined with `on_delete` set to `CASCADE`.

Complex lookups with Q objects

Field lookups using `filter()` are joined with a SQL `AND` operator. For example, `filter(field1='foo', field2='bar')` will retrieve objects where `field1` is *foo* and `field2` is *bar*. If you need to build more complex queries, such as queries with `OR` statements, you can use `Q` objects.

A `Q` object allows you to encapsulate a collection of field lookups. You can compose statements by combining `Q` objects with the `&` (and), `|` (or), and `^` (xor) operators.

For example, the following code retrieves posts with a title that starts with the string *who* or *why* (case-insensitive):

```
>>> from django.db.models import Q
>>> starts_who = Q(title__istartswith='who')
>>> starts_why = Q(title__istartswith='why')
>>> Post.objects.filter(starts_who | starts_why)
```

In this case, we use the `|` operator to build an `OR` statement.

Activate W

```

from django.db import models

class PublishedManager(models.Manager):
    def get_queryset(self):
        return super().get_queryset().filter(status = Post.Status.PUBLISHED)

# Create your models here.
class Post(models.Model):
    class Status(models.TextChoices):
        DRAFT = 'DF', 'Draft'
        PUBLISHED = 'PB', 'Published'

    objects = models.Manager()
    published = PublishedManager()
*****

```

The first manager declared in a model becomes the default manager. You can use the Meta attribute `default_manager_name` to specify a different default manager. If no manager is defined in the model, Django automatically creates the objects default manager for it. If you declare any managers for your model but you want to keep the objects manager as well, you have to add it explicitly to your model.

```

*****

(.venv) C:\Tests\j-l-django\Test-02\jlokastite>python manage.py shell
13 objects imported automatically (use -v 2 for details).

Ctrl click to launch VS Code Native REPL
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr  8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from blog.models import Post
>>> Post.published.all()
<QuerySet [<Post: Test-02>]>
>>> Post.objects.all()
<QuerySet [<Post: Test-02>, <Post: Test-01>]>
*****

```

The view for list all published posts:

```

from django.shortcuts import render
from .models import Post

# Create your views here.
def post_list(request):
    posts = Post.published.all()
    # in this way we avoid positional arguments error
    return render(request, 'blog/post/list.html', {'posts': posts});
-----

```


The view for post detail:

```
from django.shortcuts import get_object_or_404, render
from django.http import Http404
from .models import Post

def post_detail(request, id: int):
    try:
        # post = Post.publish.get(pk=id)
        post = get_object_or_404(Post, id=id, status=Post.Status.PUBLISHED)
    except Post.DoesNotExist:
        raise Http404("No post found")
    return render(
        request, 'blog/post/detail.html', {'post': post}
    )
```

The urls.py for blog-app is:

```
from django.urls import path
from . import views

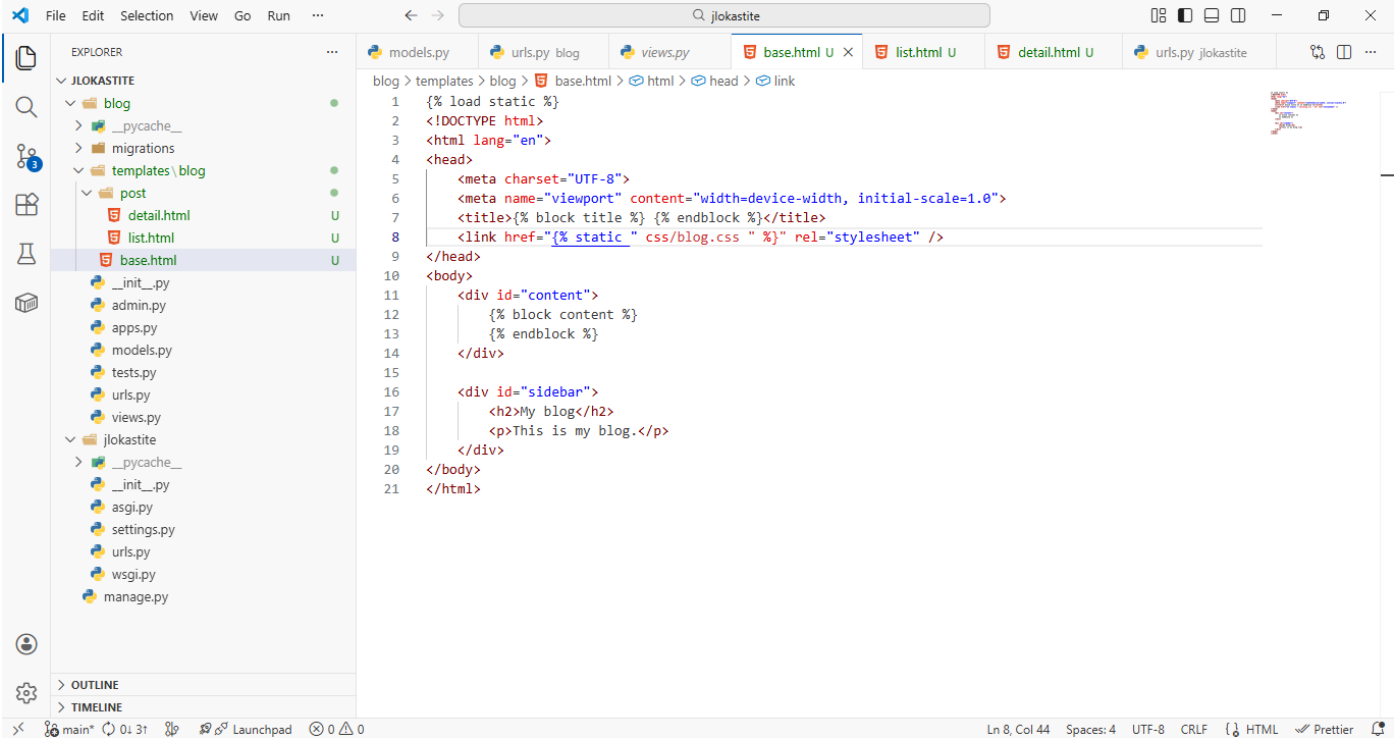
app_name = 'blog'
urlpatterns = [
    path('', views.post_list, name='post_list'),
    path('<int:id>', views.post_detail, name='post_detail'),
]
```

We define an **application namespace** with the **app_name variable**. This allows us to organize URLs by application and **use the name when referring to them**.

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('blog/', include('blog.urls', namespace='blog'))
]
```

Namespaces have to be unique across your entire project. We will refer to our blog URLs easily by using the namespace followed by a colon and the URL name, **for example**, **blog:post_list** and **blog:post_detail**.



{% load static %} tells Django to load the static template tags that are provided by the `django.contrib.staticfiles` application, which is contained in the `INSTALLED_APPS` setting. After loading them, you can use the `{% static %}` template tag throughout this template. With this template tag, you can include the static files

```

{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %} {% endblock %}</title>
    <link href="{% static 'css/blog.css' %}" rel="stylesheet" />
</head>
<body>
    <div id="content">
        {% block content %}
        {% endblock %}
    </div>

    <div id="sidebar">
        <h2>My blog</h2>
        <p>This is my blog.</p>
    </div>
</body>
</html>

```

```

{% extends "blog/base.html" %}

```

```

{% block title %} My block {% endblock %}

```

```

{% block content %}
    <h1>My Blog</h1>
    {% for post in posts %}
        <h2>
            <a href="{% url 'blog:post_detail' post.id %}">
                {{ post.title }}
            </a>
        </h2>

        <p class="date">
            Published {{ post.publish }} by {{ post.author }}
        </p>

        {{ post.body|truncatewords:30|linebreaks }}
    {% endfor %}
{% endblock %}

```

```
{% extends "blog/base.html" %}

{% block title %} {{ post.title }} {% endblock %}

{% block content %}
    <h1>{{ post.title }}</h1>
    <p class="date">
        Published {{ post.publish }} by {{ post.author }}
    </p>
    {{ post.body|linebreaks }}
{% endblock %}
```